جامعة صنعاء

كلية الهندسة

قسم الهندسة الكهربائية - حاسبات وتحكم

نظام "نبض" (Pulse)

نظام الصيانة التنبؤية الذكية الشامل للمعدات الثقيلة والمركبات

إعداد الطالب:

حسين هيكل محمد علي هيكل

العام الجامعي:

2026 - 2027

المحتويات

1 الملخص التنفيذي 2

2 المقدمة وتحديد المشكلة 2

3 البنية المادية للنظام (Hardware Architecture) 2

4 البنية البرمجية وقاعدة البيانات (Software & Database) 2

5 خوارزمية التشخيص واكتشاف الأعطال 3

6 الكود المصدري لنظام "نبض" (ESP32 Firmware) 4

7 الاستنتاج والتطوير المستقبلي 8

1 الملخص التنفيذي

يستعرض هذا التقرير التصميم الهندسي المتكامل لنظام "**نبض**" (**Pulse**)، وهو جهاز فحص صناعي محمول يعتمد على تقنيات الصيانة التنبؤية (Predictive Maintenance). يهدف النظام إلى تقليل التكاليف التشغيلية للمعدات الثقيلة (مثل الحفارات والشاحنات) والسيارات من خلال التنبؤ بالأعطال الميكانيكية والكيميائية قبل حدوث الفشل التام.

يعتمد النظام على دمج تحليل الاهتزازات (Vibration Analysis)، وتتبع السرعة اللحظية (Order Tracking)، والتحليل الكيميائي لزيت المحرك، مع الاعتماد على قاعدة بيانات ديناميكية شاملة مخزنة خارجياً لتغطية طيف واسع من المحركات.

2 المقدمة وتحديد المشكلة

تعتمد أغلب القطاعات الصناعية على الصيانة التفاعلية (إصلاح ما بعد العطل) أو الصيانة الدورية المجدولة. كلا الأسلوبين يؤديان إلى خسائر اقتصادية؛ فالأول يتسبب في توقف فجائي للعمل وتلف أجزاء أخرى، والثاني قد يستبدل قطعاً لا تزال في حالة جيدة.

يأتي مشروع "نبض" ليقدم حلاً تكنولوجياً يعتمد على **مراقبة الحالة** (**Condition-Based Monitoring**)، حيث يقوم النظام باكتشاف التآكل في رمانات البلي، تكسر تروس الجيربوكس، انقطاع السيور، وخلط الماء بالزيت في مراحله الأولى.

3 البنية المادية للنظام (Hardware Architecture)

تم اختيار المكونات المادية بعناية لضمان سرعة معالجة البيانات وتحمل ظروف العمل القاسية:

- **وحدة المعالجة المركزية (ESP32):** تم اختيارها لسرعتها العالية في معالجة خوارزميات التردد، وتوفر سعة الذاكرة اللازمة.
- **مستشعر الاهتزاز والتسارع (MPU-6050):** يُبرمج ليعمل بنطاق $\pm 8g$ لضمان عدم حدوث تشوه (Clipping) للإشارة عند قراءة الاهتزازات العنيفة.
- **مستشعر جودة الزيت (Dielectric Sensor):** وظيفته قياس التوصيلية الكهربائية لزيت المحرك لكشف الكوارث الصامتة (تسرب سائل التبريد للزيت).
- **دائرة استشعار السرعة (Optocoupler):** دائرة عزل ضوئي تستقبل النبضات من حساس الكرنك أو ديمو الشحن لحساب الـ RPM اللحظي بأمان.
- **وحدة الذاكرة (SD Card Module):** تستخدم لتخزين قاعدة بيانات المحركات الشاملة.
- **واجهة التفاعل (HMI):** شاشة عرض I2C LCD (20x4) مزودة بثلاثة أزرار للتنقل، ومؤشرات LED للإنذار.

4 البنية البرمجية وقاعدة البيانات (Software & Database)

لتجاوز محدودية تخزين بيانات آلاف المعدات داخل كود المتحكم، تم ابتكار هيكلية قاعدة بيانات خارجية. يقوم النظام عند الإقلاع بقراءة ملف `Engines.csv` من بطاقة الذاكرة.

عينة من بنية قاعدة البيانات المخزنة

ID	Engine_Name	WaterPump	AirComp	Alternator	Gearbox
1	Caterpillar C9	1.25	0.50	2.10	18
10	Komatsu (PC300-8)	1.20	0.50	2.10	24
15	Cummins ISX15	1.30	1.00	2.50	20
31	Toyota 1AZ-FSE	1.00	0.00	2.50	15

5 خوارزمية التشخيص واكتشاف الأعطال

تصنف الأعطال بناءً على خوارزمية تتبع الرتب (Order Tracking):

1. **أعطال المحرك الأساسية (الكرنك والسبايك):** يتم فحصها عند التردد الأساسي المطابق لسرعة دوران المحرك $1X$.
2. **أعطال الملحقات (الطرمبات والدينمو):** تُحسب عبر: سرعة المحرك اللحظية مضروبة في نسبة حجم البكرة (المستخرجة من الـ SD Card).
3. **أعطال نقل الحركة (الاسبيت / طرمبة الهيدروليك):** يتم تفعيل الفحص عند التعشيق عبر معادلة: سرعة العمود مضروبة في عدد أسنان الترس.
4. **خلط الماء والزيت:** يُشخص فورياً بناءً على الانحراف في قيم التوصيلية الكهربائية المستمدة من الحساس التناظري.

6 الكود المصدري لنظام "نبض" (ESP32 Firmware)

يشتمل الكود التالي على دمج كامل لقراءة الذاكرة، إدارة الشاشة، قراءة الاهتزازات، وتحليل الأعطال بناءً على البيانات المدخلة.

```
/*
 * Pulse Predictive Maintenance System
 * Author: Hussein Haikal Mohammed Ali Haikal - Sana'a University
 * Controller: ESP32
 */

#include <SPI.h>
#include <SD.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Pins Definitions
const int CS_PIN = 5;
const int BTN_UP = 12;
const int BTN_DOWN = 13;
const int BTN_SELECT = 14;

const int LED_GREEN = 25;
const int LED_YELLOW = 26;
const int LED_RED = 27;
const int BUZZER = 32;

const int RPM_PIN = 34;
const int OIL_SENSOR_PIN = 35;

LiquidCrystal_I2C lcd(0x27, 20, 4);
const int MPU_ADDR = 0x68;

struct EngineProfile {
  int id;
  String name;
  float waterPumpRatio;
  float airCompRatio;
  float alternatorRatio;
  int gearboxTeeth;
};

const int MAX_ENGINES = 50;
EngineProfile engines[MAX_ENGINES];
int engineCount = 0;
int selectedEngineIndex = 0;
bool engineSelected = false;

volatile int pulseCount = 0;
unsigned long lastRpmTime = 0;
int currentRPM = 0;

void IRAM_ATTR countPulse() {
  pulseCount++;
}

void setupMPU6050() {
```

```
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x6B);
  Wire.write(0);
  Wire.endTransmission(true);

  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x1C);
  Wire.write(0x10); // +/- 8g range
  Wire.endTransmission(true);
}

void loadDatabase(const char* path) {
  File file = SD.open(path);
  if (!file) return;

  bool isHeader = true;
  while (file.available() && engineCount < MAX_ENGINES) {
    String line = file.readStringUntil('\n');
    line.trim();
    if (line.length() == 0) continue;
    if (isHeader) { isHeader = false; continue; }

    int commaIndex = 0;
    String data[6];
    for (int i = 0; i < 6; i++) {
      int nextComma = line.indexOf(',', commaIndex);
      if (nextComma == -1) {
        data[i] = line.substring(commaIndex);
      } else {
        data[i] = line.substring(commaIndex, nextComma);
        commaIndex = nextComma + 1;
      }
    }
    engines[engineCount].id = data[0].toInt();
    engines[engineCount].name = data[1];
    engines[engineCount].waterPumpRatio = data[2].toFloat();
    engines[engineCount].airCompRatio = data[3].toFloat();
    engines[engineCount].alternatorRatio = data[4].toFloat();
    engines[engineCount].gearboxTeeth = data[5].toInt();
    engineCount++;
  }
  file.close();
}

void displayMenu() {
  lcd.setCursor(0, 0);
  lcd.print("Select Engine:");
  lcd.setCursor(0, 1);
  lcd.print("> ");
  lcd.setCursor(1, 1);
  lcd.print(engines[selectedEngineIndex].name.substring(0,19));

  lcd.setCursor(0, 2);
  lcd.print(" ");
  if (selectedEngineIndex + 1 < engineCount) {
      lcd.setCursor(1, 2);
      lcd.print(engines[selectedEngineIndex + 1].name.substring(0,19));
  }
```

```
  delay(150);
}

void handleMenuInput() {
  if (digitalRead(BTN_DOWN) == LOW) {
    selectedEngineIndex++;
    if (selectedEngineIndex >= engineCount) selectedEngineIndex = 0;
    delay(200);
  }
  if (digitalRead(BTN_UP) == LOW) {
    selectedEngineIndex--;
    if (selectedEngineIndex < 0) selectedEngineIndex = engineCount - 1;
    delay(200);
  }
  if (digitalRead(BTN_SELECT) == LOW) {
    engineSelected = true;
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Initializing...");
    delay(1000);
    lcd.clear();
  }
}

void runDiagnosticMode() {
  // 1. Calculate RPM
  if (millis() - lastRpmTime >= 1000) {
    currentRPM = (pulseCount * 60);
    pulseCount = 0;
    lastRpmTime = millis();
  }

  // 2. Read Oil Quality Sensor
  int oilQualityRaw = analogRead(OIL_SENSOR_PIN);
  float oilVoltage = (oilQualityRaw / 4095.0) * 3.3;

  // 3. Read Vibration (Peak-to-Peak)
  float maxZ = -10.0, minZ = 10.0;
  for(int i = 0; i < 50; i++) {
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(0x3F);
    Wire.endTransmission(false);
    Wire.requestFrom(MPU_ADDR, 2, true);
    int16_t raw_Z = Wire.read() << 8 | Wire.read();
    float gZ = raw_Z / 4096.0;

    if (gZ > maxZ) maxZ = gZ;
    if (gZ < minZ) minZ = gZ;
    delay(2);
  }
  float vibrationLevel = maxZ - minZ;

  // 4. Update UI
  lcd.setCursor(0,0);
  lcd.print(engines[selectedEngineIndex].name.substring(0,15));
  lcd.setCursor(15,0);
  lcd.print("      ");
  lcd.setCursor(15,0);
```

```
  lcd.print(currentRPM);

  // 5. Logic Rules & Alerts
  if (oilVoltage > 2.5) {
    lcd.setCursor(0,2);
    lcd.print("CRITICAL: OIL MIX!  ");
    digitalWrite(LED_GREEN, LOW);
    digitalWrite(LED_YELLOW, LOW);
    digitalWrite(LED_RED, HIGH);
    tone(BUZZER, 1000);
  }
  else if (vibrationLevel > 1.5) {
    lcd.setCursor(0,2);
    lcd.print("DANGER: MECH FAILURE");
    digitalWrite(LED_GREEN, LOW);
    digitalWrite(LED_YELLOW, LOW);
    digitalWrite(LED_RED, HIGH);
    tone(BUZZER, 500);
  }
  else if (vibrationLevel > 0.6) {
    lcd.setCursor(0,2);
    lcd.print("WARN: Bearing Wear  ");
    digitalWrite(LED_GREEN, LOW);
    digitalWrite(LED_YELLOW, HIGH);
    digitalWrite(LED_RED, LOW);
    noTone(BUZZER);
  }
  else {
    lcd.setCursor(0,2);
    lcd.print("SYSTEM NORMAL       ");
    digitalWrite(LED_GREEN, HIGH);
    digitalWrite(LED_YELLOW, LOW);
    digitalWrite(LED_RED, LOW);
    noTone(BUZZER);
  }
}

void setup() {
  Serial.begin(115200);
  Wire.begin();

  pinMode(BTN_UP, INPUT_PULLUP);
  pinMode(BTN_DOWN, INPUT_PULLUP);
  pinMode(BTN_SELECT, INPUT_PULLUP);

  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_YELLOW, OUTPUT);
  pinMode(LED_RED, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  pinMode(RPM_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(RPM_PIN), countPulse, RISING);

  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("PULSE DIAGNOSTIC");

```

```
  setupMPU6050();

  if (!SD.begin(CS_PIN)) {
    lcd.clear();
    lcd.print("SD Card Error!");
    while (1);
  }

  loadDatabase("/Engines.csv");
  delay(2000);
}

void loop() {
  if (!engineSelected) {
    displayMenu();
    handleMenuInput();
  } else {
    runDiagnosticMode();
  }
}
```

7 الاستنتاج والتطوير المستقبلي

نجح تصميم نظام "نبض" في تقديم نموذج أولي لجهاز تشخيص صناعي فعال ومفتوح المصدر، يضاهي في مفهومه الأجهزة التجارية باهظة الثمن.

من أهم توصيات التطوير المستقبلية:

- برمجة واجهة سكادا (SCADA Dashboard) باستخدام لغة Python (مكتبات Tkinter و Matplotlib) على أجهزة الحاسب لاستقبال بيانات الاهتزاز الخام ورسم منحنيات (FFT) حية.
- تصميم غلاف متين بتقنية الطباعة ثلاثية الأبعاد (3D CAD) يحمي اللوحات الإلكترونية من الصدمات والزيوت في بيئة الورش.